Python Basic Introduction — Interview Ready

Hinglish Deep Notes • Requested typography: 18 / 16 / 14 • 1.5 line spacing • Colourful tables & section highlights

Topics: Introduction, History, Features, Command Interpreter & IDLE, Applications, Python 2/3, Program Structure, Quotations, Indentation, Operators, Basic Data Types & Built-in Objects.

1. Python kya hai? — Basic Introduction

Definition

Python ek high-level, general-purpose programming language hai jo readable syntax, rapid development aur large ecosystem ke liye famous hai. High-level ka matlab hai ki programmer ko low-level machine details directly handle nahi karni padti.

Interview line: “Python is a high-level, general-purpose programming language known for readable syntax, rapid development and a large ecosystem.”

Why Python popular hai?

- Readable syntax: code comparatively clean aur English-like hota hai.
- Rapid development: kam boilerplate ke saath functionality build kar sakte ho.
- Huge ecosystem: web, AI/ML, data, automation, testing etc. ke liye libraries available hain.
- Cross-platform: Windows, Linux aur macOS par run kar sakti hai.
- Multiple programming styles: procedural, object-oriented aur functional approaches support karti hai.

2. History of Python

Python ko Guido van Rossum ne create kiya. Development late 1980s me start hua aur first public release 1991 me hua. Naam Monty Python comedy group se inspired tha.

- 1989 — Python development start.
- 1991 — First public release.
- 2000 — Python 2.0.
- 2008 — Python 3.0.
- 1 January 2020 — Python 2 officially EOL.

3. Features of Python

Feature	Simple Hinglish Explanation
Simple & Readable	Syntax easy to read; beginners aur teams ke liye helpful.
High-level	Memory/register level details manually handle nahi karne padte.
Dynamic typing	Variable declare karte waqt fixed type likhna mandatory nahi.
Object-oriented	Class, object, inheritance, polymorphism etc. support.
Portable	Compatible platforms par same code largely run ho sakta hai.
Open source	Free/open-source language and ecosystem.
Large standard library	Files, JSON, OS, dates, math etc. ke modules available.
Automatic memory management	Runtime memory management aur garbage collection handle karta hai.
Extensible	C/C++ jaise languages ke saath integrate/extend kiya ja sakta hai.

4. Command Interpreter / REPL

Command interpreter interactive mode provide karta hai jahan Python statements type karte hi output mil sakta hai. Isko REPL kehte hain: Read Evaluate Print Loop.

```
>>> 2 + 3
5
>>> name = "Tarun"
>>> print(name)
Tarun
```

Use: quick testing, calculations aur small experiments. Large application ke liye .py files + development environment better hai.

5. IDLE — Integrated Development and Learning Environment

IDLE Python ka basic development environment hai. Isme Python Shell aur Code Editor dono milte hain.

- Shell — interactive REPL.
- Editor — .py file create/edit/run.
- Syntax highlighting — code elements identify karne me help.
- Basic debugger — debugging support.

Interview line: “IDLE provides an interactive Python shell and editor for writing and executing Python programs.”

6. Applications of Python

Area	Application
Web Development	Django, Flask, FastAPI se websites, backend aur REST APIs.

Area	Application
Data Analysis	NumPy/Pandas se data cleaning, transformation aur analysis.
AI / ML	scikit-learn, PyTorch, TensorFlow ecosystem.
Automation	Repetitive tasks, file operations aur scripting.
Testing	Automated testing and QA workflows.
Desktop GUI	Tkinter, PyQt etc.
DevOps / Cloud	Automation, tooling aur API interaction.
Scientific Computing	Numerical computing, simulation, research.

7. Python 2 vs Python 3

Point	Python 2	Python 3
Status	EOL / discontinued	Modern Python family
print	print "Hello"	print("Hello")
Division	2/3 0 (integers)	2/3 0.666...
range()	range list; xrange available	range object; memory-efficient
Input	raw_input() string input	input() returns string
Unicode	More complex handling	str is Unicode text; bytes for binary

Interview trap: Python 2 sirf “old syntax” nahi hai; Python 3 me language semantics aur standard behavior me important changes aaye. New projects me Python 3 use karo.

8. Basic Program Structure

Python me program statements, expressions, functions, classes aur blocks se bana hota hai. C/C++/Java ki tarah { } braces se block define nahi hota; indentation block define karti hai.

```
name = "Tarun"
age = 21

if age >= 18:
print(name, "is an adult")
else:
print(name, "is a minor")
```

- Comments — # se single-line comment.
- Statements — assignment, function call, import etc.
- Expressions — values produce karne wale combinations.
- Blocks — if/for/while/def/class etc. ke indented statements.
- Functions — reusable logic ke liye def.
- Imports — modules/packages use karne ke liye.

9. Quotations in Python

String ko single quotes, double quotes ya triple quotes se represent kar sakte ho.

```
name = 'Tarun'
city = "Indore"
message = """This is
multiple line text."""
```

- '...' aur "..." normal strings ke liye.
- '''...''' / """..""" multiline strings aur docstrings ke liye.
- Quote character ko represent karne ke liye escaping use kar sakte ho.

10. Indentation — Very Important

Python me indentation syntax ka part hai. Generally 4 spaces per indentation level follow kiya jata hai.

```
if age >= 18:
print("Adult")
print("Eligible")
print("Program continues")
```

First two print statements if block ke andar hain; last print block ke bahar hai. Same block me indentation consistent rakho aur tabs/spaces mix avoid karo.

Interview answer: “Python uses indentation to define compound-statement blocks, which also improves readability.”

11. Operators in Python

Category	Operators	Purpose
Arithmetic	+ - * / // % **	Calculation
Comparison	== != > < >= <=	Values compare
Assignment	= += -= *= /= //= %= **=	Assign/update
Logical	and or not	Conditions combine/negate
Bitwise	& | ^ ~ << >>	Bits par operation
Membership	in, not in	Collection membership
Identity	is, is not	Same object identity

== vs is

```
a = [1, 2]
b = [1, 2]
print(a == b) # True: values same
print(a is b) # False: usually different objects
```

== value/equality comparison karta hai; is object identity check karta hai. Normal value comparison me is ko == ka replacement mat samjho.

12. Operator Precedence — Basic Hierarchy

Multiple operators wale expression me precedence decide karti hai kaunsa operation pehle hoga. Ambiguous expression me parentheses use karna best practice hai.

```
2 + 3 * 4 # 14
(2 + 3) * 4 # 20
```

High-level order: parentheses ** unary *, /, //, % +, - shifts bitwise comparisons not and or conditional assignment.

13. Basic Data Types

Type	Meaning	Example
int	Whole numbers	10, -5, 0
float	Decimal numbers	3.14, -0.5
complex	Complex numbers	2+3j
bool	Boolean values	True, False
str	Text / Unicode string	"Python"
NoneType	Absence of value	None

```
x = 10
print(type(x)) # int
x = "10"
print(type(x)) # str
```

Interview: 10 aur "10" same nahi hain. Pehla int object hai, doosra str object.

14. Built-in Objects / Data Structures

Object	Nature	Example	Key Point
list	Ordered + mutable	[1,2,3]	Duplicates allowed; modify possible.
tuple	Ordered + immutable	(1,2,3)	Elements change nahi kar sakte.
set	Unique collection	{1,2,3}	Duplicates removed; membership useful.
dict	Key-value mapping	{"name":"Tarun"}	Keys ke through values access.
range	Immutable range object	range(5)	Loops ke liye memory-efficient.
bytes	Immutable binary data	b"abc"	Binary data.
bytearray	Mutable binary data	bytearray(b"abc")	Mutable binary data.

15. Mutable vs Immutable

Mutable object ko existing object me modify kar sakte ho. Immutable object ko in-place modify nahi kar sakte; change ke case me new value/object binding create hoti hai.

Mutable	Immutable
list	int
dict	float
set	bool
bytearray	str
	tuple
	bytes

```
s = "abc"
s = s + "d" # new string value

lst = [1, 2]
lst.append(3) # same list modified
```

16. Variables, Objects, type() and id()

Python me variable name ek object ko refer karta hai. Isliye same variable later different type ke object ko refer kar sakta hai.

```
x = 10
print(type(x))
x = "Python"
print(type(x))
print(id(x))
```

type(x) object ka type/class batata hai. id(x) object identity ka integer identifier return karta hai; exact representation implementation-dependent ho sakti hai.

17. Interview Questions — Rapid Fire

Q. Python kya hai?

Ans: High-level, general-purpose programming language with readable syntax and large ecosystem.

Q. Python interpreted hai?

Ans: CPython source ko bytecode me compile karta hai, phir Python Virtual Machine bytecode execute karti hai; sirf “interpreted” bolna incomplete answer hai.

Q. Dynamic typing kya hai?

Ans: Variable declaration me fixed type specify nahi karna padta; object ka type runtime par hota hai.

Q. Indentation kyun?

Ans: Python grammar me blocks indentation se define hote hain aur readability improve hoti hai.

Q. Python 2 ya 3?

Ans: Python 3; Python 2 officially EOL hai.

Q. list vs tuple?

Ans: Dono ordered; list mutable, tuple immutable.

Q. set ka use?

Ans: Unique elements aur membership operations.

Q. dict kya hai?

Ans: Key-value mapping.

Q. == vs is?

Ans: == equality/value; is identity.

Q. IDLE kya hai?

Ans: Integrated Development and Learning Environment — shell + editor.

Q. 10 vs "10"?

Ans: int vs str.

18. One-Page Revision Mind Map

- Python = high-level + general-purpose + readable + huge ecosystem.
- Creator = Guido van Rossum; first public release = 1991.
- Modern Python = Python 3; Python 2 = EOL.
- REPL = Read Evaluate Print Loop.
- IDLE = Integrated Development and Learning Environment.
- Blocks = indentation; generally 4 spaces.
- Strings = single, double, triple quotes.
- Operators = arithmetic, comparison, assignment, logical, bitwise, membership, identity.
- Types = int, float, complex, bool, str, NoneType.
- Collections = list, tuple, set, dict; plus range, bytes, bytearray.
- Mutable = list, dict, set, bytearray; immutable = int, float, bool, str, tuple, bytes.
- == = value/equality; is = identity.

- type() = type/class; id() = object identity identifier.
- CPython = source bytecode Python Virtual Machine execution.

19. Interview Answer Formula

Definition Why Example Difference/Use-case is format me answer do. Example: “Python dynamically typed hai” bolne ke baad explain karo ki type explicitly declare nahi karna padta, runtime object ka type determine hota hai, aur same variable later another type ko refer kar sakta hai.

`if __name__ == "__main__":` — Definition

`if __name__ == "__main__":` Python ka *main guard* hai.

Ye check karta hai ki **current Python file directly run hui hai ya kisi doosri file mein import hui hai.**

Code Example

Python Run

```python
def hello():
    print("Hello Tarun")


if __name__ == "__main__":
    hello()
```

13032a5956e

Bilkul, **import karke ek hi example** se samjho:

`calculator.py`

Python

```python
def add(a, b):
    return a + b


if __name__ == "__main__":
    print("Calculator directly run hua")
    print(add(10, 20))
```

`main.py`

Python

```python
import calculator

print(calculator.add(5, 3))
```

Jab `main.py` run karoge:

```
8
```

`Calculator directly run hua` **print nahi hoga** ❌

Kyun?

Kyuki `calculator.py` ko humne **import** kiya hai, directly run nahi. Isliye:

```python
if __name__ == "__main__":
```

false ho jata hai.

👉 **Fayda:** Import karte time file ka extra/test code automatically execute nahi hota; sirf functions/classes use kar sakte ho.

`is` vs `==` in Python

Dono ka use **comparison** ke liye hota hai, lekin dono **alag cheez check** karte hain.

Operator	Kya check karta hai?	Simple meaning
`==`	**Value / content**	"Dono ki value same hai?"
`is`	**Object identity**	"Dono exactly same object hain?"

Ek code se samjho 👇

```python
a = [10, 20]
b = [10, 20]
c = a

print(a == b)    # True
print(a is b)    # False

print(a == c)    # True
print(a is c)    # True
```

Step-by-step

1. `a == b`

Python

```python
a = [10, 20]
b = [10, 20]
```

Dono ke andar **same values** hain:

```
a → [10, 20]
b → [10, 20]
```

Isliye:

Python

```python
a == b    # True
```

2. `a is b`

Dono ki values same hain, **lekin objects alag hain**.

```
a ──→ [10, 20]  ← alag object
b ──→ [10, 20]  ← alag object
```

Isliye:

Python

```python
a is b    # False
```

---

3. `c = a`

Ab:

Python

```python
c = a
```

ka matlab `c` **same object** ko refer karega.

513032a5956e

Async I/O in Python kya hai?

Async I/O (Asynchronous Input/Output) ka matlab hai:

> Jab Python kisi **slow I/O operation** (jaise API call, database query, file/network operation) ka wait kar raha ho, tab woh **dusra kaam** kar sakta hai, instead of simply waiting.

Simple example

Normal/synchronous code:

Python

```python
result1 = get_data()   # 5 sec wait
result2 = get_data()   # phir 5 sec wait
```

Total ≈ **10 sec**

Async code mein dono operations overlap ho sakte hain:

Async code mein dono operations overlap ho sakte hain:

```
Task 1 → wait ——————————→ complete
Task 2 → wait ——————————→ complete
```

Total ≈ **5 sec** (agar dono ka wait similar ho).

Python mein Async I/O kaise karte hain?

Python mein mainly `asyncio` module use hota hai.

Python

```python
import asyncio

async def task(name):
    print(name, "start")
    await asyncio.sleep(2)
    print(name, "complete")

async def main():
    await asyncio.gather(
        task("Task 1"),
        task("Task 2")
    )

asyncio.run(main())
```

Output:

```
Task 1 start
Task 2 start
Task 1 complete
Task 2 complete
```

Dono tasks ka wait **parallel-like/concurrently** handle hua, isliye roughly **2 seconds**, 4 seconds nahi.

Important Keywords

Keyword	Meaning
`async`	Function ko asynchronous coroutine banata hai
`await`	Kisi async operation ka wait karta hai, aur event loop ko meanwhile dusre tasks chalane deta hai
`asyncio`	Python ka asynchronous programming framework
`asyncio.run()`	Async program start karta hai
`asyncio.gather()`	Multiple async tasks ko concurrently run karne mein help karta hai
`create_task()`	Coroutine ko scheduled task banata hai

-513032a5956e

"Async nahi ho to?" 🤔

Agar async use nahi karte, to normal **synchronous/blocking** approach hoti hai:

Python

Run

```python
import time

def task(name):
    print(name, "start")
    time.sleep(2)
    print(name, "complete")

task("Task 1")
task("Task 2")
```

Approx time:

```
2 sec + 2 sec = 4 sec
```

Async version:

```
Task 1 ── wait ──┐
Task 2 ── wait ──┤ → ~2 sec
```

Async I/O ke "types" kya hote hain?

Agar aap **I/O operations ke types** pooch rahe ho, common examples:

I/O Type	Example
Network I/O	API request, HTTP
Database I/O	MySQL/MongoDB query
Socket I/O	Client-server communication
File I/O	File read/write
Subprocess I/O	Exte[illegible]command/program

JavaScript	Python	Purpose
`async function`	`async def`	Async function banana
`await`	`await`	Async operation ka wait karna
`Promise`	Coroutine / Awaitable	Future async result ko represent karna
`Promise.all()`	`asyncio.gather()`	Multiple async tasks ko concurrently run/wait karna
`Promise.race()`	`asyncio.wait(..., FIRST_COMPLETED)`	Jo task pehle complete ho, usko handle karna
`Promise.allSettled()`	`asyncio.gather(..., return_exceptions=True)`	Sabhi tasks ke result/error collect karna
`setTimeout()`	`asyncio.sleep()`	Delay ke baad kaam karna
`setInterval()`	`while + asyncio.s...)`	Repeated task chalana

dd-513032a5956e

`clearTimeout()`	`Task.cancel()`	Async task cancel karna
`clearInterval()`	`Task.cancel()`	Repeating task cancel karna
Event Loop	`asyncio` **Event Loop**	Async tasks ko schedule/manage karna
Microtask Queue	**No exact equivalent**	JS-specific queue; Python mein exact same mechanism nahi
Macrotask / Task Queue	**No exact equivalent**	JS-specific scheduling concept
`fetch()`	`aiohttp` / `httpx`	Async HTTP/API request
WebSocket	`websockets`	Async WebSocket communication
`try/catch`	`try/except`	Error handling
`finally`	`finally`	Cleanup
`AbortController`	`Task.cancel()`	Async operation cancel karna

```python
import asyncio


# 1. async def → JavaScript async function
async def task(name, seconds):

    print(name, "started")

    # 2. await + asyncio.sleep()
    #    JavaScript ke setTimeout() jaisa
    await asyncio.sleep(seconds)

    print(name, "completed")
    return name


async def main():

    # 3. create_task() → task ko schedule karta hai
    t1 = asyncio.create_task(task("Task 1", 2))
    t2 = asyncio.create_task(task("Task 1"
```

```
    t1 = asyncio.create_task(task("Task 1", 2))
    t2 = asyncio.create_task(task("Task 1", 3))

    # 4. gather() → Promise.all() jaisa
    result = await asyncio.gather(t1, t2)

    print("Result =", result)


# 5. asyncio.run() → Event Loop start
asyncio.run(main())
```

Output

```
Task 1 started
Task 1 started
Task 1 completed
Task 1 completed
Result = ['Task 1', 'Task 1']
```